DUET: Dual-Granularity Address Translation for High-Capacity SSDs

Anonymous Author(s)
Affiliation omitted for double-blind review

***Abstract*—High-capacity QLC SSDs cannot keep a 4 KB page-level mapping table in on-device DRAM. Under a limited DRAM budget, existing FTLs slow reads with mapping misses, slow writes with block-level merges, or turn small writes into read-modify-writes of a uniform coarse indirection unit (IU). We present DUET, an FTL that keeps the coarse IU and maps smaller writes as 4 KB pages in a DRAM hash table. DUET spills these entries to NAND when the table fills and merges fine pages one coarse IU at a time. With 1/8 of the page-level mapping DRAM, DUET reaches 93% of the throughput of a page-level FTL with unlimited DRAM, whereas existing FTLs reach at most 73%.**

I. Introduction

The flash translation layer (FTL) is the SSD firmware layer that maps every logical page to its physical location. Because NAND flash cannot be overwritten without erasing a whole block, physical pages are programmed sequentially within a block and every update is written out of place, so the FTL maintains a mapping table that translates each logical page to its current physical page. Most commercial SSDs deployed today map at 4 KB granularity and keep this table in on-device DRAM.

With QLC NAND, SSDs of 64 and 128 TB are being released and developed [1], [2]. At 4 KB granularity, a 64 TB SSD has 2^{34} logical pages, so each entry needs at least 34 bits and the table takes 68 GB. A 128 TB SSD needs 35-bit entries and 140 GB. Providing this much on-device DRAM is impractical because of its cost, power, and board area [3], so large SSDs have to translate addresses with a mapping DRAM budget that is only a fraction of the full page-level table (Figure 1a).

Techniques that reduce the mapping DRAM fall into two broad classes, and both pay for it in latency. *On-demand mapping* [4], [5] (Figure 1b) keeps the page-level table in flash and keeps part of it in an on-device map cache. A read that misses in the map cache must first read its mapping entry from NAND before it can read the data. This extra read is synchronous, so read latency spikes whenever the map cache misses. *Block-level hybrid mapping* [6]–[9] (Figure 1c), on the other hand, maps most data per NAND block and appends updates to page-mapped log blocks. When free space runs low, the log blocks are merged back into their data blocks, and a merge must copy all valid pages of a whole NAND block. Each 3D NAND generation enlarges the NAND block [10], so a merge copies more pages, and these copies compete with host writes for NAND bandwidth. In our measurements, the p99 write latency of FAST [7], a representative block-level hybrid mapping FTL, is 2.8–3.4× that of a page-level FTL under random I/O.

Many recent commercial high-capacity SSDs instead map all data with a *uniform coarse indirection unit* (uniform coarse IU) of 16–64 KB [3], [11]. The whole coarse table fits in on-device DRAM, with no mapping misses and no block-level merges. The cost moves to small writes. When a write is smaller than a coarse IU, the FTL reads the rest of the unit, merges the new data, and writes the whole unit again (read-modify-write, RMW). This raises the write amplification (WA) and lengthens write latency.

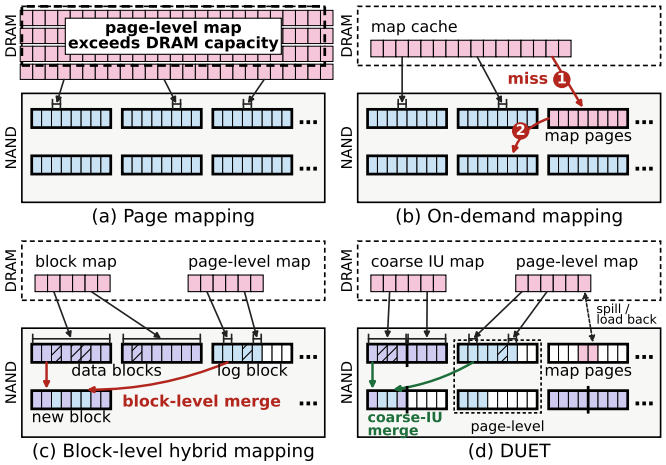


Fig. 1. Address translation of four FTLs. In (d), page-level mappings spill to map pages instead of being merged when the DRAM table is full, and are loaded back on access.

We present **DUET** (**DU**al-granularity **E**conomical **T**ranslation), an FTL that keeps two mapping granularities, the coarse IU and the 4 KB page, under a fixed mapping DRAM budget (Figure 1d). Most data are mapped per coarse IU, and recent writes smaller than a coarse IU are mapped at 4 KB granularity. They are written out of place as 4 KB pages, and the corresponding pages of the coarse IU become invalid, so the host write path has no RMW. Most of the page-level mappings stay in on-device DRAM, and DUET uses NAND only to stage a small part of them so that they fit the DRAM budget, which is fixed in hardware. DUET thus needs far fewer mapping reads than on-demand mapping, which reduces its read latency. When either the page-level or the coarse-IU-level area runs out of space, its garbage collection (GC) merges the 4 KB pages of a coarse IU into it and rewrites the whole coarse IU. A merge rewrites only one coarse IU regardless of the NAND block size, so DUET never performs the block-level merges of existing hybrid mapping FTLs.

We implement DUET and six other FTLs in NVMeVirt [12], a kernel-level NVMe SSD emulator, and evaluate them under a limited mapping-DRAM budget with microbenchmarks, application workloads, and real-world traces. DUET reaches 93% of the throughput of a page-level FTL with unlimited DRAM, whereas on-demand mapping, block-level hybrid mapping, and a uniform coarse IU reach at most 73%, 57%, and 66%, respectively. Its average read latency is up to 2.4× lower than that of on-demand mapping, and its p99 write latency is 4.6× lower than that of block-level hybrid mapping on TPC-C.

This paper makes the following contributions:

- An analysis showing that existing FTLs fit a limited mapping DRAM budget only by paying in read latency (on-demand mapping), merge cost and tail latency (block-level hybrid mapping), or RMW (uniform coarse IU).
- The design of DUET, which combines dual-granularity mapping

with coarse-IU-level merging.
- An implementation of DUET and six other FTLs in a common emulator. Under the same DRAM budget, DUET achieves the highest average throughput among the bounded-DRAM FTLs, within 7% of a page-level FTL with unlimited DRAM.

II. Background and Motivation

A. Address Translation under a DRAM Budget

NAND flash is read and programmed in pages (4–16 KB) but erased only in much larger blocks, and a page cannot be overwritten before its block is erased. The SSD controller therefore writes every update out of place and runs a flash translation layer (FTL) that maps each logical page to its current physical page. Old copies become invalid, and garbage collection (GC) reclaims a block by copying its remaining valid pages elsewhere and erasing it. These copies make the number of pages written to NAND flash larger than the amount of host writes. The ratio is the write amplification (WA), and spare over-provisioned (OP) capacity keeps it bounded.

A page-level FTL keeps one entry per 4 KB logical page, so its table is about 0.1% of the capacity and grows linearly with it. Conventional commercial SSDs keep the whole table in on-device DRAM, but high-capacity QLC SSDs can hold only a fraction of it because of the cost, power, and board area that so much DRAM would take [3]. The FTL must then either keep part of the mapping in flash or map data in larger units. The three approaches below take these two routes, and each one moves the cost to a different part of the I/O path.

On-demand mapping. On-demand mapping stores the page-level table in non-volatile NAND flash as flash translation pages (FTPs) and keeps only recently used entries in an on-device *map cache*, as introduced by DFTL [4]. A small global translation directory (GTD) locates each FTP. A read that misses in the map cache must first read the FTP and only then the data page, so two flash reads are serialized on the read path. Evicting a dirty entry also writes its FTP back, which adds write amplification. LeaFTL [5] compresses the map cache into learned linear segments, so the same DRAM covers more pages. The segments, however, are built only from runs of sequentially placed pages, and a mispredicted location imposes an extra flash read. μ-FTL [13] maps data at multiple granularities with variable-size extents, but it keeps the extents in a B+-tree in flash and caches only part of the tree in DRAM. For randomly written data, these FTLs miss often, and every miss turns one read into two dependent flash reads. Read latency therefore depends on the map-cache hit ratio, not on the device.

Block-level hybrid mapping. Block-level hybrid FTLs such as BAST (block-associative sector translation) [6], FAST (fully-associative sector translation) [7], and Janus-FTL [8] map most data per NAND block, which needs only one entry per block, and absorb updates in a small pool of page-mapped *log blocks*. When the pool runs out, a log block is reclaimed by *merging*. The valid pages of a logical block are copied from its data block and the log block into a free block, and the old data block and log block are erased. A merge imposes a higher overhead than the GC of a page-level FTL. Page-level GC picks the block with the fewest valid pages, so it copies few pages per reclaimed block. A merge instead rewrites the whole data block even when only a few of its pages were updated, and one log block can hold updates to many logical blocks, all of which must be merged before it is erased.

The merge copies compete with host writes for NAND bandwidth, which raises write latency and lowers write throughput. Each merge also rewrites a whole NAND block, and NAND blocks keep growing with 3D NAND generations [10], so the work behind a single merge grows with them. CoMap [14] selects page or block mapping for each write request with reinforcement learning, but the data that it maps per block are still reorganized at NAND-block granularity.

Uniform coarse-IU mapping. High-capacity SSDs in production simply map all data with a *uniform coarse indirection unit* (uniform coarse IU) of 16–64 KB [3], [11]. One entry per coarse IU shrinks the table by the coarse-IU size over 4 KB, so the whole table fits in DRAM: every read takes a single flash access, and there are no block-level merges. The cost moves to writes smaller than a coarse IU. The FTL must read the rest of the unit, merge the new data, and program the whole unit again (read-modify-write, RMW). The extra reads lengthen the write path, and the rewritten pages raise write amplification by up to the coarse-IU size over the write size. In our evaluation the 1/8 budget allows a 32 KB coarse IU (4 B per 32 KB), which raises WA by 2.7–3.3× over a page-level FTL on YCSB-A and YCSB-B.

B. Motivation

Figure 2 compares the three approaches with a page-level FTL that keeps its whole table in DRAM. All FTLs run in NVMeVirt [12] with the same NAND timing and the same 1/8 mapping-DRAM budget. Each bar is the geometric mean over the 10 workloads of Section V (synthetic raw I/O, real-world traces, and applications). WA and write latency leave out YCSB-C, which is read-only.

On-demand mapping slows reads. With mapping misses, DFTL needs 1.46 flash reads per host read and LeaFTL 1.22 (Figure 2a). Their average read latency is 2.4× and 1.7× that of the page-level FTL (Figure 2b) because each miss adds a synchronous flash read before the data read.

Block-level hybrid mapping and uniform coarse IUs slow writes. FAST's block-level merges raise WA by 2.6× (Figure 2c), so its average write latency is 2.4× that of the page-level FTL (Figure 2d). The uniform coarse IU avoids merges, but every 4 KB write becomes an RMW of its 32 KB unit, so its WA grows by 2.4× and its write latency by 2.1×.

DUET, shown for reference, uses the same DRAM budget as the other bounded FTLs. With fewer flash reads per host read, its average read latency is 38–57% lower than that of on-demand mapping. With a lower WA, its average write latency is 47–54% lower than that of block-level hybrid mapping and the uniform coarse IU.

Goal. An FTL that fits a limited DRAM budget should therefore (i) minimize mapping reads from flash, so that a host read costs a single flash read, (ii) eliminate block-level merges, and (iii) absorb small writes without RMW. DUET meets all three by keeping the coarse IU and adding a fine-grained level within the same DRAM budget.

III. DUET Design

DUET maps every logical page at two mapping granularities (Figure 3). A *coarse map* keeps one entry per coarse IU, and a *fine map* maps the 4 KB fine pages written for parts of a coarse IU. The corresponding pages of the coarse IU, which are mapped by the coarse map, then become invalid. The coarse map is always in DRAM, so most reads need no flash read to find their mapping. Part of the fine map can move to NAND, but unlike demand-paging FTLs [4], [5], [13], most of the map stays in DRAM, so far fewer reads need a second flash read. Unlike hybrid FTLs [6], [7], which merge one block at a time, DUET merges fine pages one coarse IU at a time, so each merge costs less. Unlike coarse-IU FTLs [3], [11], DUET writes small writes as fine pages without RMW.

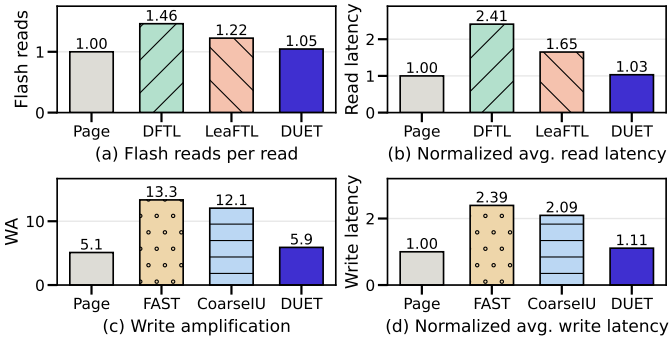


Fig. 2. Existing FTLs with about 1/8 of the page-level mapping DRAM, geometric mean over 10 workloads, and 9 for WA and write latency (YCSB-C is read-only). Latency is normalized to Page. (a) Flash reads per host read. (b) Average read latency. (c) Write amplification. (d) Average write latency.

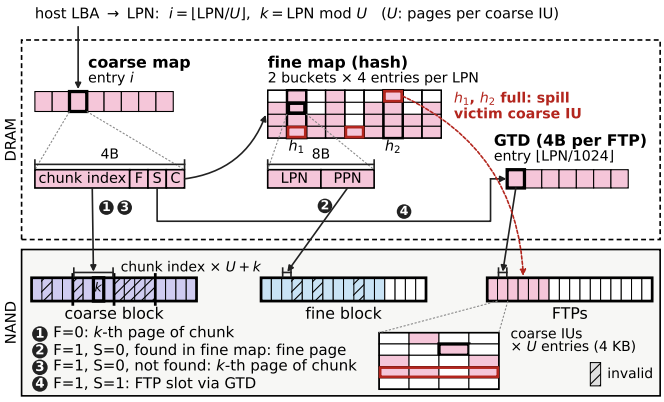


Fig. 3. DUET mapping structures.

A. Two Mapping Granularities

Coarse map. The logical space is divided into coarse IUs of U pages ($4U$ KB), so a logical page number (LPN) belongs to coarse IU $i = \lfloor \mathrm{LPN}/U \rfloor$ at page offset $k = \mathrm{LPN} \bmod U$. Each coarse IU is stored as a *chunk* of U physically consecutive, U-page-aligned pages in a *coarse block*, and its 4 B coarse entry holds the chunk index. Since a coarse entry covers U pages and is as large as a page-level entry, the coarse map takes $1/U$ of the DRAM of a page-level table.

Fine map. A write smaller than a coarse IU is written out of place as 4 KB *fine pages* into separate *fine blocks*. The fine map is a two-choice bucketized hash table [15]. With two candidate buckets, a bucket fills up much later than in an ordinary hash table with a single candidate bucket [15], so DUET evicts entries less often. Each 8 B entry holds an LPN and the physical page number (PPN) of its fine page. Each LPN has two candidate buckets of four entries each, and a new entry goes to the less loaded one. The table takes all of the DRAM budget that the coarse map leaves.

The table is similar to cuckoo hashing [16] in that each entry has two candidate buckets. Unlike cuckoo hashing, DUET never moves an existing entry to its other bucket to make room, so an insert does not trigger a chain of moves. When both candidate buckets are full, DUET instead writes the fine-map entries of a victim coarse IU to a *fine translation page* (FTP) in NAND and frees them from the table (*spill*, Section III-D). A table lookup therefore checks at most the eight entries of the two candidate buckets, and an insert writes into a free entry of one of them. A spilled entry has a fixed location in NAND, so one map-page read returns it.

Flag bits. Each coarse entry also has three flag bits. The *has-fine-pages* (F) bit tells whether the coarse IU has fine pages, the *spilled* (S) bit tells whether their fine-map entries are in NAND, and the *clean* (C) bit tells whether the entries in the table are identical to their copy in NAND (Section III-D).

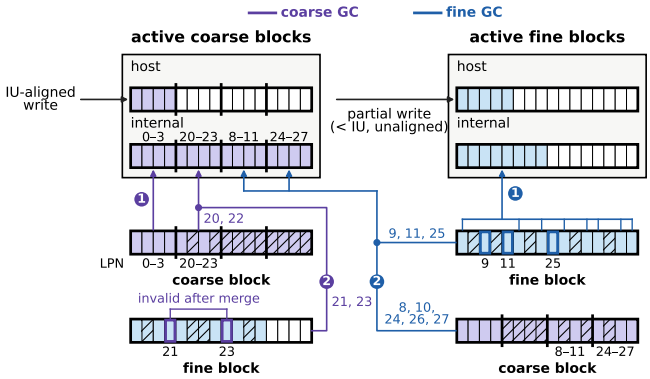


Fig. 4. DUET write and reclamation paths. Coarse GC (purple) and fine GC (blue) either (1) copy valid data or (2) merge it into new chunks.

B. Host I/O Path

Read path. A read first checks the coarse entry of coarse IU i. If F is set and S is clear, DUET looks up the LPN in the fine map. If S is set, the global translation directory (GTD) [4] locates the FTP of the LPN, and DUET reads its slot. When the fine map or the slot points to a fine page, DUET reads the fine page. Otherwise, including when F is clear, it reads page k of the chunk. A read thus reads at most three DRAM locations, the coarse entry and the two candidate buckets. A 32 B bucket lies within one DRAM row, so each location costs one random DRAM access [17]. Only a read of a spilled coarse IU adds one map-page read.

Write path. Host writes go to two *active blocks*, an active coarse block for chunks and an active fine block for fine pages. A host write is split at coarse-IU boundaries. Each IU-aligned write is written as a new chunk in the active coarse block. The old chunk and all fine pages of the coarse IU become invalid, and the corresponding fine-map entries are removed. If the coarse IU already has fine pages, DUET instead writes the IU-aligned write as fine pages while the fine blocks are below their cap (Section III-C).

A *partial write*, which is smaller than a coarse IU, is written as fine pages into the active fine block. For each fine page, DUET inserts its LPN and PPN into one of the two candidate buckets of the LPN, the less loaded one. The page of the chunk with the same LPN becomes invalid. If the LPN already has a fine page, DUET updates the PPN of its fine-map entry instead, and the old fine page becomes invalid.

DUET keeps fine pages and chunks in different blocks. Fine pages hold recently rewritten data and chunks hold colder data, and mixing data of different update frequencies in one NAND block raises the GC cost [18]–[20]. As in other FTLs [21], DUET also separates host writes from internal writes (GC and merges), so it keeps four active blocks (Figure 4). When an active block is full, DUET takes a free block as the new active block.

C. Reclaiming Path

DUET reclaims a block when the free blocks run low. Its written blocks are divided into fine blocks, which hold fine pages, and coarse blocks, which hold chunks. DUET thus has two kinds of GC. *Fine GC* reclaims a fine block, and *coarse GC* reclaims a coarse block. To find the valid data of a victim, DUET keeps a valid bitmap with one bit per chunk slot of the coarse blocks and one bit per page of the fine blocks.

Coarse-or-fine victim selection. DUET picks a victim with fewer valid pages to copy, whether it is a coarse block or a fine block. The number of fine blocks, however, is limited by the OP space. A fine

page makes one page of a chunk invalid, but the chunk still takes its place in a coarse block. The coarse blocks must therefore be able to cover the whole logical capacity, and the fine blocks can only use the space beyond it. When the fine blocks reach this cap, DUET thus reclaims a fine block. Otherwise, it compares the coarse and fine blocks and runs GC on the side whose block has fewer valid pages.

Coarse GC. Coarse GC handles each valid chunk of the victim in one of two ways (purple lines in Figure 4). A chunk whose coarse IU has no fine pages (LPN 0–3) is copied unchanged to a new chunk (1). A chunk whose coarse IU has fine pages (LPN 20–23) is only partly valid, so DUET merges it (2). DUET looks up the LPNs of the coarse IU in the fine map. An LPN found there has a fine page (LPN 21 and 23), and an LPN not found there is still valid in the chunk (LPN 20 and 22). DUET reads these pages and writes them as one merged chunk. The fine pages then become invalid, their fine-map entries are removed, and the F bit of the coarse IU is cleared.

Fine GC. Fine GC handles each coarse IU with valid fine pages in the victim in one of two ways (blue lines in Figure 4). DUET either copies the fine pages to another fine block (1) or merges the coarse IU with its chunk (2). In Figure 4, LPN 9 and 11 are merged with the chunk of LPN 8–11, and LPN 25 with the chunk of LPN 24–27. Let d be the number of fine pages of the coarse IU in all fine blocks, not only in the victim. A merge reads the chunk and these d fine pages and writes one new chunk. All d fine pages then become invalid, and their fine-map entries are removed.

Copy-or-merge decision. DUET chooses one of the two ways by a cost-benefit model. The cost is the number of pages written, and the benefit is the number of pages freed in the fine blocks. Let P be the number of pages per block, v the number of valid pages in the victim, and U the number of pages in a coarse IU. For the same victim, copying costs v page writes and frees $P - v$ pages. A merge of a coarse IU with d fine pages instead costs U page writes, one coarse IU, and frees its d fine pages. The cost-benefit value, that is, the number of pages freed in the fine blocks per page written, is thus $(P-v)/v$ for copying (1) and d/U for a merge (2). DUET copies the fine pages when $(P - v)/v \geq d/U$, and otherwise merges the coarse IU. DUET makes this decision for every coarse IU that has valid fine pages in the victim fine block.

Fine-area size. The number of fine blocks and the number of fine-map entries change at runtime with these two decisions. When coarse-or-fine victim selection often picks a coarse block, fine pages accumulate in the fine blocks, and the number of fine blocks grows up to the OP limit. When it often picks a fine block, the number of fine blocks shrinks. The copy-or-merge decision then sets the number of fine-map entries. Copying keeps the fine pages and their fine-map entries, so the fine map grows as new small writes arrive, whereas a merge moves the fine pages into a chunk and frees their fine-map entries, so the fine map shrinks. The size of the fine map thus adapts to the workload (Section V).

D. Fine-Map Spill and Load

Under a fixed DRAM budget, the table cannot index every fine page that the fine blocks can hold. For example, a budget of 128 MB per TB of capacity, 1/8 of the page-level table, holds the whole map of an SSD with uniform 32 KB coarse IUs. If DUET uses 64 KB chunks under this budget, the coarse map takes 64 MB per TB. Since each fine-map entry takes 8 B, the remaining 64 MB holds about 8M entries, which map 32 GB of fine pages. If the fine blocks use all of the 7% OP space, however, they can hold 70 GB of fine pages per TB. The table thus has no room for more than half of their fine-map entries. The DRAM capacity is fixed when the SSD hardware is designed. Unlike the FTL, which can be updated with the firmware, it cannot be changed easily.

Another challenge is hash collisions within the fine map. An insert can find both of its candidate buckets full even when other buckets still have free slots. Hashing schemes that place a colliding entry in another bucket [16], [22]–[24] make a lookup probe more buckets or an insert move existing entries, so they need more DRAM accesses. On the other hand, merging the coarse IU whose entries are pushed out of the full buckets into the coarse map causes extra NAND writes.

DUET instead keeps the fine-map entries of some coarse IUs in FTPs in NAND, so the fine blocks can use all of the OP space without more DRAM. The fine-map entries of a coarse IU move to its FTP when the table fills (*spill*) and back to the table when the coarse IU is accessed again (*load*), and no data moves in either direction. In theory, the spilled entries map at most 7% of the data, the size of the OP space, because most data are mapped by the coarse map, which is always in DRAM.

Spill. As shown in Figure 3, the fine-map entry of each LPN has a fixed 4 B slot in an FTP, which holds the entries of 1024 consecutive LPNs. As in DFTL, the GTD locates each FTP with one 4 B entry, charged to the same DRAM budget. When an insert finds both candidate buckets full, DUET picks a victim among the coarse IUs that have an entry in the two buckets. It writes all entries of the victim to its FTP, sets its S bit, and frees its table entries. The other coarse IUs of the same FTP are spilled with the same program.

Load. An access to a spilled coarse IU reads its FTP and loads only the entries of that coarse IU into the table. The other coarse IUs of the FTP stay spilled. A write loads the entries and then updates them. A read reads the FTP before the data, loads the entries, and sets the C bit of the coarse IU. The coarse IU stays clean until one of its entries changes.

What to spill. DUET picks a clean coarse IU first to reduce FTP writes to NAND. A clean coarse IU is simply dropped from the table and marked spilled because its FTP copy is still valid. If there is no clean coarse IU, DUET picks the coarse IU with the most fine pages, so one FTP program frees the most table slots.

GC. When GC moves fine pages of spilled coarse IUs, it reads and programs each FTP at most once per victim block. A merge of a spilled coarse IU reads its entries from the FTP and then clears its F and S bits.

IV. Implementation

We implement DUET and six other FTLs in NVMeVirt [12], a kernel-level NVMe SSD emulator that models the latency of every NAND read, program, and erase over its channels and dies. We add a common FTL interface to NVMeVirt. Each FTL implements the host read, write, and flush handlers and its own GC, and it issues every data and mapping access as a flash operation. Mapping reads and programs therefore take NAND time like data accesses and are counted in WA. All FTLs share the same NAND model, write buffer, and host interface and differ only in the FTL. Based on the NVMeVirt code, we also let host reads suspend programs and erases and pace background NAND operations to mimic real SSDs. These changes apply equally to all FTLs. DUET takes about 1,600 lines of C.

V. Evaluation

A. Experimental Setup

Setup. We run all experiments on a server with an Intel Xeon Gold 6342 CPU and Linux kernel 6.8. The emulated SSD has eight channels with two dies each, and its OP space is 7% of the physical capacity. For each workload, the device size is set to fit the footprint of the workload, from 22 GB to 97 GB, and all FTLs use the same

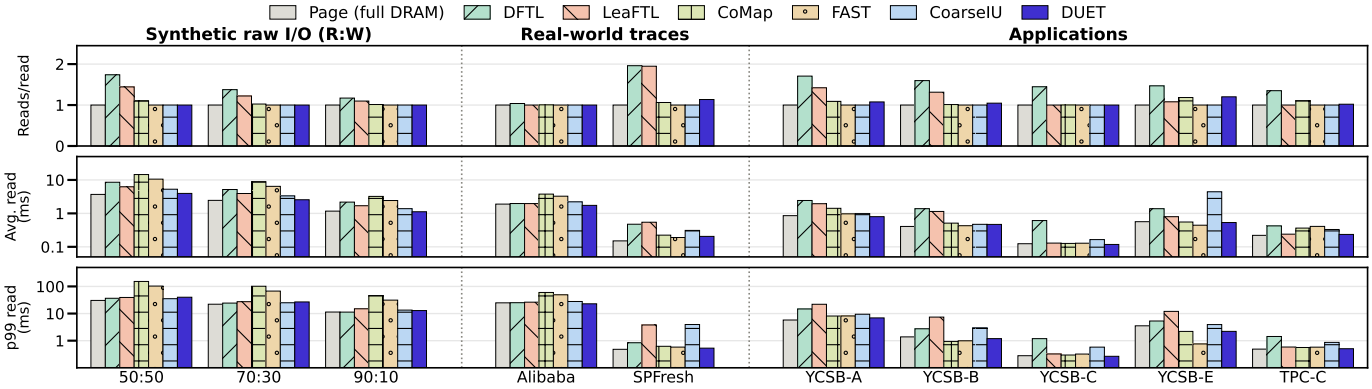


Fig. 5. Read path. From top, flash reads per host read (data plus mapping reads), and average and p99 read latency.

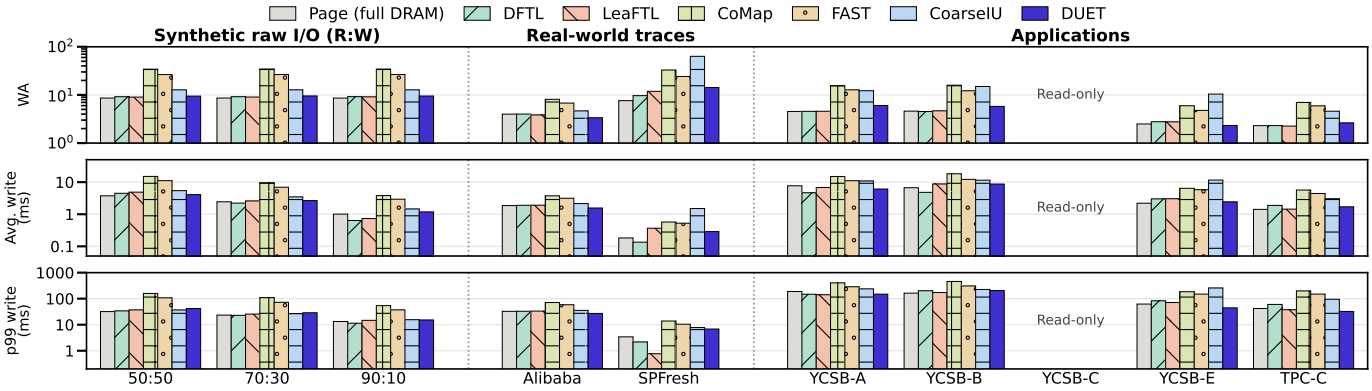


Fig. 6. Write path. From top, device write amplification, and average and p99 write latency. YCSB-C is read-only.

device size. We cap the mapping DRAM of every bounded FTL at the DRAM of a uniform 32 KB coarse IU, an IU size used in production high-capacity SSDs [3], [11]. On the 44 GB device, for example, it is 5.6 MB, whereas Page uses 43 MB. We run three types of workloads, synthetic raw I/O, real-world traces, and applications.

Workloads. The synthetic raw I/O issues uniformly random requests over the whole device. Of the requests, 70% are 4 KB and 30% are 128 KB, each aligned to its size, and the read ratio is 50%, 70%, or 90%. It writes twice the device capacity. The real-world traces are the Alibaba cloud block traces [25] and an I/O trace of SPFresh [26], a vector search index with in-place updates. From the Alibaba traces, we take the volumes whose drive writes per day are 0.1 to 1, which write 9 TB in total. We then keep 2.3% of their 1 MB regions with the full access stream of each region [27], which leaves 189 GB of writes and 76 GB of reads on a 60 GB device. From the SPFresh trace, we keep 50% of the 1 MB regions in the same way, which leaves 60 GB of writes and 1.2 TB of reads on a 22 GB device. The synthetic raw I/O and the traces are replayed on the device with 32 outstanding requests, and we measure the second half of each replay. The applications are YCSB workloads A, B, C, and E [28] and TPC-C [29] on PostgreSQL and ext4, each with 64 client threads. They run in a memory cgroup of 8.8 GB, 20% of the 44 GB device, which limits the page cache.

Baselines. We compare DUET with six other FTLs, which cover the three classes of Section II. **Page** maps every 4 KB page and keeps the whole table in DRAM, without a DRAM limit. **DFTL** [4] caches mapping entries in an LRU map cache and writes back the dirty entries of a translation page together. **LeaFTL** [5] learns linear segments, each of which predicts the physical pages of consecutive LPNs, caches them in DRAM, and sorts writes by LPN to form longer segments. **FAST** [7] maps data blocks per NAND block and appends updates to one sequential and several random log blocks, and we limit the number of log blocks so that their page-level maps fit the DRAM budget. **CoMap** [14] chooses page-level or block-level mapping for each write request with a Q-learning agent. **CoarseIU** maps all data with a uniform 32 KB coarse IU and keeps the whole table in DRAM. Its DRAM sets the budget that every other bounded FTL receives.

B. Read Latency

Figure 5 shows the read path. DUET reads the mapping from NAND only for spilled coarse IUs, so it needs 1.05 flash reads per host read on average and at most 1.20, on YCSB-E. DFTL and LeaFTL miss in their map caches and need up to 1.96 and 1.95 flash reads on SPFresh. As a result, the average read latency of DUET is within 3% of Page on average, whereas that of DFTL and LeaFTL is 2.41× and 1.65× that of Page. On YCSB-B, DUET reads 2.4× faster than LeaFTL, the faster of the two. FAST, CoMap, and CoarseIU need no mapping reads, but their reads wait behind merges and RMW. On the 50:50 synthetic raw I/O, the average read latency of FAST and CoMap is 2.9× and 3.9× that of Page. DUET also keeps its p99 read latency within 2% of Page on average. Its largest gap is on SPFresh. All its requests are 4 KB and spread over the data with little locality, so most coarse IUs get fine pages and many of them are spilled. Reads to these coarse IUs add 0.14 mapping reads per host read, and the average read latency of DUET is 1.36× that of Page.

C. Write Amplification and Write Latency

Figure 6 shows the write path. YCSB-C is read-only, so it is left out. DUET writes small writes out of place as fine pages, so its WA stays close to that of Page, with a geometric mean of 1.16× Page. FAST and CoMap merge whole blocks, and their WA reaches 3.1× and 4.0× that of Page on the synthetic raw I/O. CoarseIU rewrites a

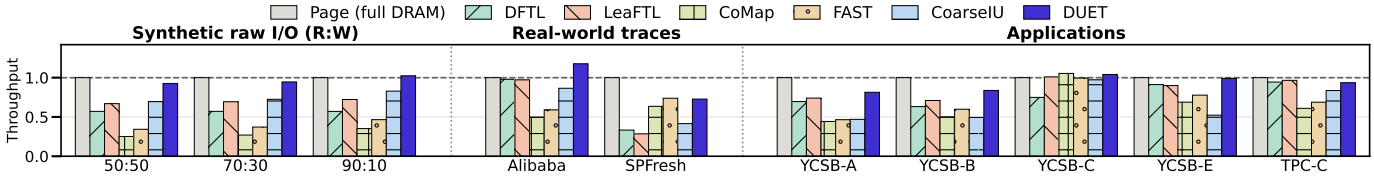


Fig. 7. Throughput normalized to Page.

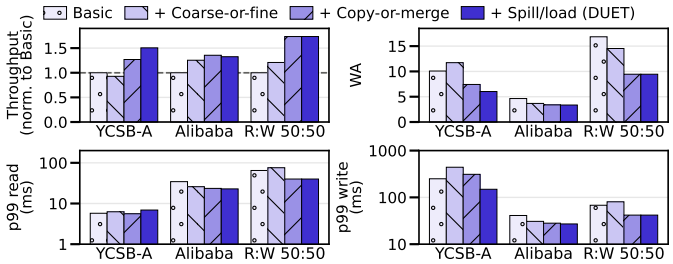


Fig. 8. Ablation. From left to right, each bar adds one more technique to Basic, and the rightmost bar is DUET. Throughput is normalized to Basic.

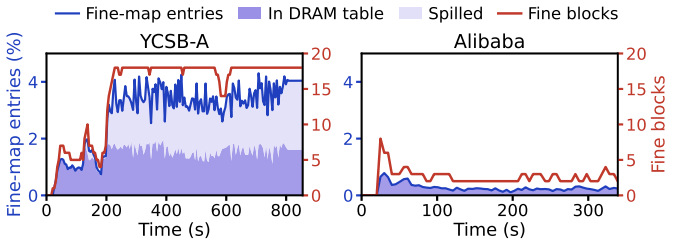


Fig. 9. Fine-map entries, as a percentage of all logical pages, and fine blocks of DUET over time, sampled every 5 s.

32 KB unit for every smaller write, so its WA is 8.3× that of Page on SPFresh, none of whose writes covers a whole 32 KB unit. As a result, the average write latency of CoarseIU, FAST, and CoMap is 2.1–3.1× that of Page, whereas that of DUET stays close to that of Page. On TPC-C, the p99 write latency of DUET is 4.6× lower than that of FAST. DFTL and LeaFTL also write out of place at 4 KB granularity and keep their write latency close to that of Page, but they pay for it on the read path.

The WA of DUET is 1.9× that of Page on SPFresh, but it is lower than that of Page on Alibaba and YCSB-E. In these workloads, small writes tend to be hot. DUET writes the writes smaller than a coarse IU to fine blocks, apart from the coarse blocks, so it separates hot data from cold data as data placement schemes do [21], [30], [31]. Page writes all data to the same active block, so hot and cold pages are mixed in a block and its GC copies more valid pages.

D. Throughput

Figure 7 shows the throughput normalized to Page. DUET reaches 93% of the throughput of Page on average, whereas LeaFTL, DFTL, CoarseIU, FAST, and CoMap reach 73%, 67%, 66%, 57%, and 49%. On Alibaba, DUET is 18% faster than Page because its WA is 16% lower. DUET is the slowest relative to Page on SPFresh, at 73%, where its higher WA and the mapping reads of spilled coarse IUs both cost it. FAST reaches 74% on SPFresh.

We also ran μ-FTL [13], which maps variable-size extents, under the same DRAM budget and leave it out of the figures. Small random writes break its extents into short ones, so its B+-tree outgrows the DRAM, and it needs 1.49 flash reads per host read. Its WA is 4.0× that of Page, and it reaches only 33% of the throughput of Page.

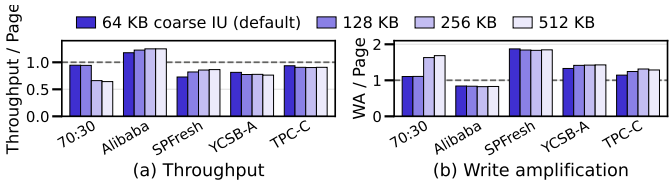


Fig. 10. DUET with coarse IUs of 64–512 KB under the same DRAM budget, normalized to Page.

E. Ablation Study

Figure 8 adds the techniques of DUET one by one to Basic on YCSB-A, Alibaba, and the 50:50 synthetic raw I/O. Basic runs coarse GC until the fine blocks reach the OP limit, merges every coarse IU in a fine-GC victim, and merges a coarse IU instead of spilling when both candidate buckets are full. Basic thus keeps its fine blocks at the limit almost all the time. Coarse-or-fine victim selection reclaims a fine block when it has fewer valid pages, which cuts the average number of fine blocks from 13–18 to 1–3. This raises the throughput by 25% on Alibaba and 21% on 50:50, but lowers it by 7% on YCSB-A, where merging every coarse IU of a fine-GC victim raises WA from 10.1 to 11.7. The copy-or-merge decision then copies fine pages when copying frees more fine-block space per page written. The fine area grows again, and the throughput rises to 1.27×, 1.36×, and 1.74× that of Basic. Spill and load matter only when the fine map fills. On YCSB-A, spilling instead of merging raises the throughput from 1.27× to 1.51× that of Basic and halves the p99 write latency from 311 ms to 149 ms. Alibaba and 50:50 never spill, so their results barely change.

F. Fine-Area Size over Time

Figure 9 shows how the fine area of DUET changes over time. Fine-map entries are shown as a percentage of all logical pages. The fine area grows and shrinks with the workload. On YCSB-A, the fine blocks grow to the limit of 18, and spilled entries reach 2.6% of the logical pages. On Alibaba, the fine area peaks at 8 blocks and 0.8% of the logical pages early in the run and then shrinks to 2–3 blocks and 0.2–0.3% without any spill.

G. Coarse-IU Size

Figure 10 varies the coarse IU of DUET from 64 KB to 512 KB under the same DRAM budget. A larger coarse IU shrinks the coarse map and leaves more DRAM to the fine map, so fewer fine-map entries spill. On SPFresh, the mapping reads per host read drop from 0.14 to 0.02, and the throughput rises from 73% to 86% of Page. A larger coarse IU, however, turns more writes into fine pages. On 70:30, the 128 KB writes still cover a 128 KB coarse IU, but become fine pages from 256 KB on. Its WA rises from 1.1× to 1.6× that of Page, and its throughput drops from 94% to 66%. The coarse IU thus works best when it is no larger than the large writes of the workload.

VI. Conclusion

DUET maps most data with coarse IUs and small writes with 4 KB fine pages, spills fine-map entries to NAND when its DRAM table fills, and merges fine pages one coarse IU at a time. With about 1/8 of the page-level mapping DRAM, DUET reaches 93% of the throughput of a page-level FTL with unlimited DRAM.

REFERENCES

[1] Solidigm, "Solidigm extends AI portfolio leadership with the introduction of 122TB drive, the world's highest capacity PCIe SSD," Press release, Nov. 2024.

[2] AnandTech, "Samsung's 128 TB-class BM1743 enterprise SSD displayed at FMS 2024," https://www.anandtech.com/show/21526, Aug. 2024.

[3] L. Bert, "Real life workloads allow more efficient data granularity and enable very large SSD capacities," Micron Technology blog, Sep. 2023.

[4] A. Gupta, Y. Kim, and B. Urgaonkar, "DFTL: A flash translation layer employing demand-based selective caching of page-level address mappings," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009.

[5] J. Sun, S. Li, Y. Sun, C. Sun, D. Vucinic, and J. Huang, "LeaFTL: A learning-based flash translation layer for solid-state drives," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[6] J. Kim, J. M. Kim, S. H. Noh, S. L. Min, and Y. Cho, "A space-efficient flash translation layer for CompactFlash systems," *IEEE Transactions on Consumer Electronics*, 2002.

[7] S.-W. Lee, D.-J. Park, T.-S. Chung, D.-H. Lee, S. Park, and H.-J. Song, "A log buffer-based flash translation layer using fully-associative sector translation," *ACM Transactions on Embedded Computing Systems*, 2007.

[8] H. Kwon, E. Kim, J. Choi, D. Lee, and S. H. Noh, "Janus-FTL: Finding the optimal point on the spectrum between page and block mapping schemes," in *Proceedings of the ACM International Conference on Embedded Software (EMSOFT)*, 2010.

[9] D. Park, B. Debnath, and D. H. C. Du, "CFTL: A convertible flash translation layer adaptive to data access patterns," in *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, 2010.

[10] C.-Y. Liu, J. B. Kotra, M. Jung, and M. T. Kandemir, "PEN: Design and evaluation of partial-erase for 3D NAND-based high density SSDs," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2018.

[11] Solidigm and Wiwynn, "Platform optimization for performance and endurance," Solidigm solution brief, Aug. 2025.

[12] S.-H. Kim, J. Shim, E. Lee, S. Jeong, I. Kang, and J.-S. Kim, "NVMeVirt: A versatile software-defined virtual NVMe device," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2023.

[13] Y.-G. Lee, D. Jung, D. Kang, and J.-S. Kim, "μ-FTL: A memory-efficient flash translation layer supporting multiple mapping granularities," in *Proceedings of the ACM International Conference on Embedded Software (EMSOFT)*, 2008.

[14] Y. Zhou, F. Wang, and D. Feng, "How to manage the mapping table of large-capacity solid-state drives," in *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, 2026.

[15] Y. Azar, A. Z. Broder, A. R. Karlin, and E. Upfal, "Balanced allocations," *SIAM Journal on Computing*, vol. 29, no. 1, pp. 180–200, 1999.

[16] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[17] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory access scheduling," in *Proceedings of the 27th International Symposium on Computer Architecture (ISCA)*, 2000, pp. 128–138.

[18] B. Van Houdt, "On the necessity of hot and cold data identification to reduce the write amplification in flash-based SSDs," *Performance Evaluation*, vol. 82, pp. 1–14, 2014.

[19] B. Van Houdt, "Performance of garbage collection algorithms for flash-based solid state drives with hot/cold data," *Performance Evaluation*, vol. 70, no. 10, pp. 692–703, 2013.

[20] J.-U. Kang, J. Hyun, H. Maeng, and S. Cho, "The multi-streamed solid-state drive," in *Proceedings of the USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2014.

[21] Q. Wang, J. Li, P. P. C. Lee, T. Ouyang, C. Shi, and L. Huang, "Separating data via block invalidation time inference for write amplification reduction in log-structured storage," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2022.

[22] A. Kirsch and M. Mitzenmacher, "The power of one move: Hashing schemes for hardware," *IEEE/ACM Transactions on Networking*, vol. 18, no. 6, pp. 1752–1765, 2010.

[23] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Proceedings of the International Symposium on Distributed Computing (DISC)*, ser. LNCS, vol. 5218, 2008, pp. 350–364.

[24] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin hood hashing," in *Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, 1985, pp. 281–288.

[25] J. Li, Q. Wang, P. P. C. Lee, and C. Shi, "An in-depth analysis of cloud block storage workloads in large-scale production," in *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, 2020.

[26] Y. Xu *et al.*, "SPFresh: Incremental in-place update for billion-scale vector search," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[27] C. A. Waldspurger, N. Park, A. Garthwaite, and I. Ahmad, "Efficient MRC construction with SHARDS," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2015.

[28] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2010.

[29] Transaction Processing Performance Council, "TPC benchmark C standard specification, revision 5.11," 2010.

[30] S. Oh, J. Kim, S. Han, J. Kim, S. Lee, and S. H. Noh, "MiDAS: Minimizing write amplification in log-structured systems through adaptive group number and size configuration," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2024, pp. 259–275.

[31] J. Kim, S. Oh, J. Kim, J. Park, J. Kim, S. Lee, and S. H. Noh, "DOGI: Data placement with oracle-guided insights for log-structured systems," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2026, pp. 543–559.